

[Sign in](#)[VasaviDappili / NumPy-Assignment](#) Public[Code](#)[Issues](#)[Pull requests](#)[Actions](#)[Projects](#)[main](#)[Go to file](#)[Code](#)[VasaviDappili](#) Update README.md

940c2d7 · 2 hours ago



README.md

Update README.md

2 hours ago

[README](#)

NumPy-Assignment

Question 1 - Student Marks Array

The marks obtained by five students in a subject are given as [78, 65, 89, 56, 92]. Create a NumPy array and display the array along with its basic properties.

Code

```
import numpy as np

marks = np.array([78, 65, 89, 56, 92])

print("Marks:", marks)
print("Dimensions:", marks.ndim)
print("Shape:", marks.shape)
print("Size:", marks.size)
print("Data type:", marks.dtype)
```



Output

```
In [1]: import numpy as np

marks = np.array([78, 65, 89, 56, 92])

print("Marks:", marks)
print("Dimensions:", marks.ndim)
print("Shape:", marks.shape)
print("Size:", marks.size)
print("Data type:", marks.dtype)

Marks: [78 65 89 56 92]
Dimensions: 1
Shape: (5,)
Size: 5
Data type: int32
```

Question 2 - Student Marks Access

The marks of five students are stored in a NumPy array as [72, 85, 64, 90, 76]. Write a program to access and display specific student marks using NumPy indexing and slicing.

Code

```
import numpy as np

marks = np.array([72, 85, 64, 90, 76])

print("All marks:", marks)
print("First student:", marks[0])
print("Third student:", marks[2])
print("Last student:", marks[-1])
print("First three students:", marks[0:3])
print("Second to fourth students:", marks[1:4])
```



Output

```
In [2]: import numpy as np

marks = np.array([72, 85, 64, 90, 76])

print("All marks:", marks)
print("First student:", marks[0])
print("Third student:", marks[2])
print("Last student:", marks[-1])
print("First three students:", marks[0:3])
print("Second to fourth students:", marks[1:4])

All marks: [72 85 64 90 76]
First student: 72
Third student: 64
Last student: 76
First three students: [72 85 64]
Second to fourth students: [85 64 90]
```

Question 3 - Subject-wise Marks

The marks obtained by five students in three subjects are given below. Create a NumPy array to represent the data and reshape it into an appropriate matrix format. [78, 85, 90, 65, 72, 80, 88, 91, 84, 56, 62, 70, 95, 89, 92]

Code

```
import numpy as np

marks = np.array([
    78, 85, 90, 65, 72,
    80, 88, 91, 84, 56,
    62, 70, 95, 89, 92
])

matrix = marks.reshape(5, 3)

print("Subject-wise marks:")
print(matrix)
```



Output

```
In [3]: import numpy as np

marks = np.array([
    78, 85, 90, 65, 72,
    80, 88, 91, 84, 56,
    62, 70, 95, 89, 92
])

matrix = marks.reshape(5, 3)

print("Subject-wise marks:")
print(matrix)

Subject-wise marks:
[[78 85 90]
 [65 72 80]
 [88 91 84]
 [56 62 70]
 [95 89 92]]
```

Question 4- Internal and External Marks

The internal and external examination marks of five

students are stored in two NumPy arrays. Write a program to calculate the final marks of each student using NumPy array operations.

Code

```
import numpy as np

internal = np.array([25, 28, 22, 24, 27])
external = np.array([65, 60, 70, 58, 68])

final_marks = internal + external

print("Internal marks:", internal)
print("External marks:", external)
print("Final marks:", final_marks)
```

Output

```
In [4]: import numpy as np

internal = np.array([25, 28, 22, 24, 27])
external = np.array([65, 60, 70, 58, 68])

final_marks = internal + external

print("Internal marks:", internal)
print("External marks:", external)
print("Final marks:", final_marks)

Internal marks: [25 28 22 24 27]
External marks: [65 60 70 58 68]
Final marks: [90 88 92 82 95]
```

Question 5- Pass Percentage Analysis

The marks obtained by five students are [45, 78, 56, 32, 91]. Using NumPy Boolean masking, identify the students who have secured 50 marks or above.

Code

```
import numpy as np

marks = np.array([45, 78, 56, 32, 91])
```

```
passed = marks >= 50

print("All marks:", marks)
print("Pass condition:", passed)
print("Students scoring 50 or above:", marks[passed])
```

Output

```
In [5]: import numpy as np

marks = np.array([45, 78, 56, 32, 91])

passed = marks >= 50

print("All marks:", marks)
print("Pass condition:", passed)
print("Students scoring 50 or above:", marks[passed])

All marks: [45 78 56 32 91]
Pass condition: [False  True  True False  True]
Students scoring 50 or above: [78 56 91]
```

Question 6 - Average Marks

The marks of five students in three subjects are represented using a NumPy matrix. Write a program to calculate the average marks of each student.

Code

```
import numpy as np

marks = np.array([
    [78, 85, 90],
    [65, 72, 80],
    [88, 91, 84],
    [56, 62, 70],
    [95, 89, 92]
])

average = np.mean(marks, axis=1)

print("Student marks:")
print(marks)
print("Average marks of each student:", average)
```

Output

```
In [6]: import numpy as np

marks = np.array([
    [78, 85, 90],
    [65, 72, 80],
    [88, 91, 84],
    [56, 62, 70],
    [95, 89, 92]
])

average = np.mean(marks, axis=1)

print("Student marks:")
print(marks)
print("Average marks of each student:", average)

Student marks:
[[78 85 90]
 [65 72 80]
 [88 91 84]
 [56 62 70]
 [95 89 92]]
Average marks of each student: [84.33333333 72.33333333 87.66666667 62.66666667 92. ]
```

Question 7 - Class Performance Statistics

The marks obtained by five students are [67, 82, 91, 74, 58]. Using NumPy statistical functions, determine the total, average, highest, lowest, and standard deviation of the marks.

Code

```
import numpy as np

marks = np.array([67, 82, 91, 74, 58])

print("Marks:", marks)
print("Total:", np.sum(marks))
print("Average:", np.mean(marks))
print("Highest marks:", np.max(marks))
print("Lowest marks:", np.min(marks))
print("Standard deviation:", round(np.std(marks), 2))
```

Output

```
In [7]: import numpy as np

marks = np.array([67, 82, 91, 74, 58])

print("Marks:", marks)
print("Total:", np.sum(marks))
print("Average:", np.mean(marks))
print("Highest marks:", np.max(marks))
print("Lowest marks:", np.min(marks))
print("Standard deviation:", round(np.std(marks), 2))

Marks: [ 67  82  91  74  58]
Total: 372
Average: 74.4
Highest marks: 91
Lowest marks: 58
```

Question 8 - Subject-wise Performance

The marks of five students in three subjects are stored in a NumPy matrix. Write a program to calculate the total marks obtained in each subject using an appropriate axis operation.

Code

```
import numpy as np

marks = np.array([
    [78, 85, 90],
    [65, 72, 80],
    [88, 91, 84],
    [56, 62, 70],
    [95, 89, 92]
])

subject_total = np.sum(marks, axis=0)

print("Student marks:")
print(marks)
print("Total marks in each subject:", subject_total)
```

Output

```
In [8]: import numpy as np

marks = np.array([
    [78, 85, 90],
    [65, 72, 80],
    [88, 91, 84],
    [56, 62, 70],
    [95, 89, 92]
])

subject_total = np.sum(marks, axis=0)

print("Student marks:")
print(marks)
print("Total marks in each subject:", subject_total)

Student marks:
[[78 85 90]
 [65 72 80]
 [88 91 84]
 [56 62 70]
 [95 89 92]]
Total marks in each subject: [382 399 416]
```

Question 9 -Student-wise Performance

The marks of five students in three subjects are stored in a NumPy matrix. Write a program to calculate the total marks obtained by each student using an appropriate axis operation.

Code

```
import numpy as np

marks = np.array([
    [78, 85, 90],
    [65, 72, 80],
    [88, 91, 84],
    [56, 62, 70],
    [95, 89, 92]
])

student_total = np.sum(marks, axis=1)

print("Student marks:")
print(marks)
print("Total marks of each student:", student_total)
```

Output

```
In [9]: import numpy as np

marks = np.array([
    [78, 85, 90],
    [65, 72, 80],
    [88, 91, 84],
    [56, 62, 70],
    [95, 89, 92]
])

student_total = np.sum(marks, axis=1)

print("Student marks:")
print(marks)
print("Total marks of each student:", student_total)

Student marks:
[[78 85 90]
 [65 72 80]
 [88 91 84]
 [56 62 70]
 [95 89 92]]
Total marks of each student: [253 217 263 188 276]
```

Question 10-Student Ranking

The total marks obtained by five students are [245, 278, 219, 290, 256]. Use NumPy sorting and indexing operations to arrange the marks in order and determine the ranking of the students.

Code

```
import numpy as np

marks = np.array([245, 278, 219, 290, 256])

sorted_marks = np.sort(marks)[::-1]
ranking = np.argsort(marks)[::-1]

print("Original marks:", marks)
print("Marks in descending order:", sorted_marks)
print("Student positions by rank (1-based):", ranking + 1)

for i in range(len(ranking)):
    student = ranking[i] + 1
    print("Rank", i + 1, "- Student", student,
          "- Marks:", marks[ranking[i]])
```

Output

```
In [10]: import numpy as np

marks = np.array([245, 278, 219, 290, 256])

sorted_marks = np.sort(marks)[::-1]
ranking = np.argsort(marks)[::-1]

print("Original marks:", marks)
print("Marks in descending order:", sorted_marks)
print("Student positions by rank (1-based):", ranking + 1)

for i in range(len(ranking)):
    student = ranking[i] + 1
    print("Rank", i + 1, "- Student", student,
          "- Marks:", marks[ranking[i]])

Original marks: [245 278 219 290 256]
Marks in descending order: [290 278 256 245 219]
Student positions by rank (1-based): [4 2 5 1 3]
Rank 1 - Student 4 - Marks: 290
Rank 2 - Student 2 - Marks: 278
Rank 3 - Student 5 - Marks: 256
Rank 4 - Student 1 - Marks: 245
Rank 5 - Student 3 - Marks: 219
```

Question 11 - Duplicate Marks Analysis

The marks obtained by five students are [85, 92, 85, 76,

92]. Use NumPy functions to identify the unique marks obtained by the students.

Code

```
import numpy as np

marks = np.array([85, 92, 85, 76, 92])

unique_marks = np.unique(marks)

print("Original marks:", marks)
print("Unique marks:", unique_marks)
```



Output

```
In [11]: import numpy as np

marks = np.array([85, 92, 85, 76, 92])

unique_marks = np.unique(marks)

print("Original marks:", marks)
print("Unique marks:", unique_marks)

Original marks: [85 92 85 76 92]
Unique marks: [76 85 92]
```

Question 12- Missing Marks

The marks of five students are represented as [78, 85, np.nan, 92, 67], where np.nan represents a missing mark. Write a NumPy program to calculate the average marks without considering the missing value.

Code

```
import numpy as np

marks = np.array([78, 85, np.nan, 92, 67])

average = np.nanmean(marks)

print("Marks:", marks)
print("Average without missing marks:", average)
```



Output

```
In [12]: import numpy as np

marks = np.array([78, 85, np.nan, 92, 67])

average = np.nanmean(marks)

print("Marks:", marks)
print("Average without missing marks:", average)

Marks: [78. 85. nan 92. 67.]
Average without missing marks: 80.5
```

Question 13- Grade Classification

The marks obtained by five students are [95, 82, 74, 61, 45]. Using NumPy conditional operations, classify the students into appropriate grade categories based on their marks.

Code

```
import numpy as np

marks = np.array([95, 82, 74, 61, 45])

grades = np.select(
    [
        marks >= 90,
        marks >= 80,
        marks >= 70,
        marks >= 60
    ],
    ["A", "B", "C", "D"],
    default="Fail"
)

print("Marks:", marks)
print("Grades:", grades)
```

Output

```
In [13]: import numpy as np

marks = np.array([95, 82, 74, 61, 45])
```

```
grades = np.select(
    [
        marks >= 90,
        marks >= 80,
        marks >= 70,
        marks >= 60
    ],
    ["A", "B", "C", "D"],
    default="Fail"
)

print("Marks:", marks)
print("Grades:", grades)

Marks: [95 82 74 61 45]
Grades: ['A' 'B' 'C' 'D' 'Fail']
```

Question 14- Random Marks Generation

Generate marks for five students using NumPy's random number generation functionality. Perform basic statistical analysis on the generated marks.

Code

```
import numpy as np

np.random.seed(42)

marks = np.random.randint(0, 101, size=5)

print("Random marks:", marks)
print("Total marks:", np.sum(marks))
print("Average marks:", np.mean(marks))
print("Highest marks:", np.max(marks))
print("Lowest marks:", np.min(marks))
```

Output

```
In [14]: import numpy as np

np.random.seed(42)

marks = np.random.randint(0, 101, size=5)

print("Random marks:", marks)
print("Total marks:", np.sum(marks))
print("Average marks:", np.mean(marks))
print("Highest marks:", np.max(marks))
print("Lowest marks:", np.min(marks))

Random marks: [51 92 14 71 60]
Total marks: 288
Average marks: 57.6
Highest marks: 92
Lowest marks: 14
```

Question 15-Student Performance Analysis

The marks of five students in three subjects are stored in a NumPy array. Develop a program to perform a complete student performance analysis by calculating the total marks, average marks, highest marks, lowest marks, and identifying students who perform above the class average.

Code

```
import numpy as np

marks = np.array([
    [78, 85, 90],
    [65, 72, 80],
    [88, 91, 84],
    [56, 62, 70],
    [95, 89, 92]
])

total = np.sum(marks, axis=1)
average = np.mean(marks, axis=1)
highest = np.max(marks, axis=1)
lowest = np.min(marks, axis=1)

class_average = np.mean(average)

above_average = np.where(average > class_average)[0] + 1

print("Student marks:")
print(marks)
print("Total marks:", total)
print("Average marks:", np.round(average, 2))
print("Highest marks:", highest)
print("Lowest marks:", lowest)
print("Class average:", round(class_average, 2))
print("Students above class average:", above_average)
```

Output

```
In [15]: import numpy as np
marks = np.array([
```

```
marks = np.array([[
    [78, 85, 90],
    [65, 72, 80],
    [88, 91, 84],
    [56, 62, 70],
    [95, 89, 92]
]])

total = np.sum(marks, axis=1)
average = np.mean(marks, axis=1)
highest = np.max(marks, axis=1)
lowest = np.min(marks, axis=1)

class_average = np.mean(average)

above_average = np.where(average > class_average)[0] + 1

print("Student marks:")
print(marks)
print("Total marks:", total)
print("Average marks:", np.round(average, 2))
print("Highest marks:", highest)
print("Lowest marks:", lowest)
print("Class average:", round(class_average, 2))
print("Students above class average:", above_average)

Student marks:
[[78 85 90]
 [65 72 80]
 [88 91 84]
 [56 62 70]
 [95 89 92]]
Total marks: [253 217 263 188 276]
Average marks: [84.33 72.33 87.67 62.67 92. ]
Highest marks: [90 80 91 70 95]
Lowest marks: [78 65 84 56 89]
Class average: 79.8
Students above class average: [1 3 5]
```

Releases

No releases published

Contributors 1



VasaviDappili

[Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#) [Manage cookies](#)

Do not share my personal information



© 2026 GitHub, Inc.